

Microsoft Entra ID (Azure AD) Single Sign-On Setup Guide

Expense Management Platform

Configuration reference for enabling enterprise SSO

Version 1.0 · June 2026

This guide walks an Azure administrator through every step required to enable Microsoft Entra ID single sign-on for the Expense Management platform: registering the application, configuring redirect URIs, defining app roles, exposing the API, and wiring the resulting identifiers into the frontend (Next.js / Auth.js) and backend (FastAPI) configuration. No code changes are required — SSO is enabled entirely through configuration.

1. How Authentication Works

The platform authenticates through a standard OpenID Connect (OIDC) Authorization-Code + PKCE flow:

1. The user clicks “Sign in with Microsoft” on the portal login page.
2. Auth.js (NextAuth v5) redirects the browser to Microsoft Entra ID, which authenticates the user and returns an authorization code.
3. Auth.js exchanges the code for an ID token + access token (PKCE protects the exchange).
4. The frontend forwards the access token to the backend API.
5. The FastAPI backend validates the token against Entra’s JWKS (signature, issuer, audience), then resolves the user’s application role and agency from its own database (“DB-by-email” JIT provisioning).

Key design point: Entra handles authentication (who you are); the application database remains the source of truth for authorization (role + agency). Switching the platform from local password login to Entra SSO is a single configuration flip — `AUTH_PROVIDER=entra` on the backend plus the Entra client credentials on the frontend.

2. Prerequisites

- An Azure subscription with a Microsoft Entra ID tenant.
- Permission to register applications (Application Administrator or Global Administrator).
- The deployed (or local) URLs of the portal frontend and API backend.
- Ability to set environment variables / Key Vault secrets in your hosting environment.

This guide is written for local development first (the app is not yet deployed), so the examples use localhost. The production hosts are shown alongside for when you deploy — the same Entra app registration supports both at once (register multiple redirect URIs).

Component	Local (now)	Production (later)
Portal (frontend)	http://localhost:3000	https://expenses.yourcompany.com
API (backend)	http://localhost:8000	https://api.expenses.yourcompany.com

Note — localhost is special: Entra allows plain http for localhost (every other host must be https), so you can test SSO locally without a TLS certificate. The redirect URI uses the portal host on port 3000 (not the API on 8000), because Auth.js on the frontend handles the OIDC callback. Use localhost, not 127.0.0.1 — Entra treats them as different origins.

3. Register the Application in Entra ID

3.1 Create the app registration

6. Sign in to the Azure Portal → Microsoft Entra ID → App registrations → New registration.
7. Name: “Expense Management Platform” (any descriptive name).

8. Supported account types: choose “Accounts in this organizational directory only” (single tenant) unless you intend multi-tenant / external access.

Redirect URI: select platform **Web** and add the Auth.js callback path. For local development use:

```
http://localhost:3000/api/auth/callback/microsoft-entra-id
```

When you deploy, add the production URL too (the same app can hold both):

```
https://expenses.yourcompany.com/api/auth/callback/microsoft-entra-id
```

⚠ **The callback path** `/api/auth/callback/microsoft-entra-id` is fixed by Auth.js — it must match exactly (scheme, host, port, path). Use the portal host (port 3000), not the API (8000); http is permitted because it is localhost.

9. Click Register. On the Overview page, copy the Application (client) ID and Directory (tenant) ID — you will need both.

3.2 Create a client secret

10. In the app registration → Certificates & secrets → Client secrets → New client secret.
11. Add a description and an expiry (e.g. 12 or 24 months). Note the expiry — SSO breaks silently when a secret expires.
12. Copy the secret Value immediately (it is shown only once). Store it in Key Vault, never in source control.

3.3 Configure tokens & ID token claims

13. App registration → Token configuration → Add optional claim → ID → add email and (optionally) family_name, given_name.
14. If prompted, enable Microsoft Graph permissions to surface these claims (email, profile).

This ensures the `email` claim is present — the backend uses it to match the user to an existing account.

4. Define Application Roles (Authorization)

The platform has four roles: employee, manager, finance, admin. You can manage roles two ways — pick the one that matches your governance model:

Option A — Database-driven (recommended, default). Entra only authenticates; the user’s role is whatever your application database says (matched by email). No app roles needed in Entra. New users are denied until an administrator provisions them (safest).

Option B — Entra app roles. Define roles in Entra and assign users to them; the backend maps the Entra role to an application role for brand-new (JIT) users. Useful if you want Entra to be the role authority for first-time sign-ins.

4.1 Create app roles (Option B only)

15. App registration → App roles → Create app role. Create one per role:

Display name	Value (Entra)	Maps to app role
Employee	Expense.Employee	employee
Manager	Expense.Manager	manager
Finance Approver	Expense.Finance	finance
Administrator	Expense.Admin	admin

Allowed member types: Users/Groups. After creating, go to Enterprise applications → your app → Users and groups to assign people to roles.

The Entra-value → app-role mapping is configured on the backend via `APP_OIDC_ROLE_MAP` (see §6).

5. Expose the API (Audience)

The backend validates that tokens were minted for this application (the audience check). The simplest, robust configuration is to use the application's own client ID as the audience.

- Easiest path: set the backend audience to the Application (client) ID from §3.1. Auth.js requests tokens for that client, so the audience matches.
- Optional (custom API scope): App registration → Expose an API → Set an Application ID URI (e.g. `api://<client-id>`) → Add a scope (e.g. `access_as_user`). Use this if you want a dedicated API resource with its own scopes.

6. Backend Configuration (FastAPI)

Set these environment variables on the API. They map to `api/app/config.py` (all backend settings use the `APP_` prefix). In Azure, project them from Key Vault via Managed Identity — never hard-code secrets.

Variable	Value	Purpose
<code>APP_AUTH_PROVIDER</code>	entra	The single switch that turns on Entra SSO.
<code>APP_ENTRA_TENANT_ID</code>	<Directory (tenant) ID>	From the app registration Overview.
<code>APP_ENTRA_AUDIENCE</code>	<Application (client) ID>	The

		audience the token must target (see §5).
APP_OIDC_AUTO_PROVISION	false	true = auto-create unknown users; false = deny until provisioned (safest).
APP_OIDC_DEFAULT_ROLE	employee	Role for auto-provisioned users with no mapping (only if auto-provision = true).
APP_OIDC_DEFAULT_AGENCY_ID	<agency id>	Agency for auto-provisioned users (empty = deny).
APP_OIDC_ROLE_MAP	{"Expense.Admin":"admin","Expense.Finance":"finance","Expense.Manager":"manager"}	Entra app-role value → app role (Option B).

Note: when **APP_AUTH_PROVIDER=entra**, the backend automatically derives the OIDC issuer and JWKS URL from the tenant ID, so **APP_ENTRA_JWKS_URL** / **APP_OIDC_ISSUER** do not need to be set manually for standard Entra tenants.

Example backend `.env` (or Key Vault references):

```
APP_AUTH_PROVIDER=entra
APP_ENTRA_TENANT_ID=00000000-1111-2222-3333-444444444444
APP_ENTRA_AUDIENCE=aaaaaaaa-bbbb-cccc-dddd-eeeeeeeeeeeeee
APP_OIDC_AUTO_PROVISION=false
APP_CORS_ALLOW_ORIGINS=http://localhost:3000      # production:
https://expenses.yourcompany.com
```

7. Frontend Configuration (Next.js / Auth.js)

Set these on the portal. They map to `frontend/src/auth.ts`. Entra SSO is auto-enabled as soon as the client ID and issuer are present.

Variable	Value	Purpose
AUTH_MICROSOFT_ENTRA_ID_ID	<Application (client) ID>	From the app registration Overview.
AUTH_MICROSOFT_ENTRA_ID_SECRET	<client secret value>	From Certificates & secrets (§3.2).
AUTH_MICROSOFT_ENTRA_ID_ISSUER	https://login.microsoftonline.com/<tenant-id>/v2.0	Tenant-specific OIDC issuer.
AUTH_SECRET	<random 32+ byte string>	Encrypts the session cookie. Generate with: openssl rand -base64 32
NEXT_PUBLIC_API_BASE_URL	http://localhost:8000	Where the frontend calls the backend (local; prod = https API host).
AUTH_OIDC_TOKEN_URL	https://login.microsoftonline.com/<tenant-id>/oauth2/v2.0/token	Used for silent token refresh (optional but recommended).
AUTH_OIDC_CLIENT_ID	<Application (client) ID>	Same client ID (for refresh).
AUTH_OIDC_CLIENT_SECRET	<client secret value>	Same secret (for refresh).

Example frontend `.env.local`:

```
AUTH_SECRET=replace-with-openssl-rand-base64-32
AUTH_MICROSOFT_ENTRA_ID_ID=aaaaaaaa-bbbb-cccc-dddd-eeeeeeeeeeee
AUTH_MICROSOFT_ENTRA_ID_SECRET=your-client-secret-value
AUTH_MICROSOFT_ENTRA_ID_ISSUER=https://login.microsoftonline.com/00000000-1111-2222-3333-444444444444/v2.0
NEXT_PUBLIC_API_BASE_URL=http://localhost:8000
AUTH_OIDC_TOKEN_URL=https://login.microsoftonline.com/00000000-1111-2222-3333-444444444444/oauth2/v2.0/token
AUTH_OIDC_CLIENT_ID=aaaaaaaa-bbbb-cccc-dddd-eeeeeeeeeeee
AUTH_OIDC_CLIENT_SECRET=your-client-secret-value
```

Only `NEXT_PUBLIC_API_BASE_URL` and `APP_CORS_ALLOW_ORIGINS` change between local and production. The `login.microsoftonline.com` issuer/token URLs are Microsoft's own endpoints and stay `https` in every environment.

8. User Provisioning

Because authorization lives in the application database, each user must exist there with the correct role and agency before (or at) first sign-in:

- Pre-provision (recommended): an administrator creates the user records (matched by the email Entra returns). Sign-in then “just works” and the existing role/agency apply.
- JIT auto-provision: set `APP_OIDC_AUTO_PROVISION=true` so unknown users are created on first sign-in using the mapped Entra role (or `APP_OIDC_DEFAULT_ROLE`) and `APP_OIDC_DEFAULT_AGENCY_ID`.

Identity matching order: (1) Entra object ID if previously linked → (2) email match (links the Entra subject to the account) → (3) JIT create or deny. For existing users, the database role always wins.

9. Validate the Setup

16. Deploy/restart both apps with the new variables.
17. Open the portal login page — a “Sign in with Microsoft” option should appear.
18. Sign in with a test user. You should be redirected to Microsoft, authenticate, and land on the role-appropriate dashboard.
19. Confirm the role/agency are correct (the database value drives what the user sees).
20. Sign out and confirm the session ends; protected routes redirect unauthenticated users to `/login`.

Quick backend check: call the API with the Entra access token and confirm the resolved identity:

```
curl -H "Authorization: Bearer <entra-access-token>" \
http://localhost:8000/auth/me
```

10. Troubleshooting

Symptom	Cause / Fix
AADSTS50011: redirect URI mismatch	The redirect URI in Entra must exactly equal <code>/api/auth/callback/microsoft-entra-id</code> on your portal host (scheme, host, path).
Login succeeds but user lands on /login	AUTH_SECRET missing or changed → session cookie can't be decrypted. Set a stable AUTH_SECRET.
401 from the API after login	Audience mismatch – APP_ENTRA_AUDIENCE must equal the client ID the token was minted for; tenant ID must be correct.
“Contact your administrator” on first sign-in	User has no DB account and APP_OIDC_AUTO_PROVISION=false. Provision the user or enable JIT.
Wrong role after sign-in	Existing DB role wins. Update the user's role in the database, or use Entra app roles + APP_OIDC_ROLE_MAP for JIT users.
SSO breaks after some months	The client secret expired. Create a new secret (§3.2) and update AUTH_MICROSOFT_ENTRA_ID_SECRET / AUTH_OIDC_CLIENT_SECRET.
No “Sign in with Microsoft” button	Frontend missing AUTH_MICROSOFT_ENTRA_ID_ID and/or _ISSUER – both are required to register the provider.

11. Security & Production Notes

- Store all secrets (client secret, AUTH_SECRET) in Azure Key Vault; project them via Managed Identity. Never commit them.
- Use HTTPS everywhere; set the real portal origin in APP_CORS_ALLOW_ORIGINS (no wildcards in production).
- Keep APP_OIDC_AUTO_PROVISION=false unless you explicitly want open self-provisioning.
- Track client-secret expiry; rotate before it lapses. Consider certificate credentials for long-lived deployments.
- The backend independently validates every token (signature/issuer/audience) and enforces RBAC — authorization is never trusted from the frontend.
- offline_access scope is requested so the session can silently refresh; sessions are capped at 8h with a 30-minute idle timeout.